#1 Find the closest point to a given target. We also pass the best values found so far for nearest neighbor (*NN*) and its distance, to help pruning. These values default to `null`, `infinity` for a call on the tree root.

#2 If `node` is `null`, we are traversing an empty tree, so the nearest neighbor can't change with respect to what we have already found.

#3 Otherwise, we have 3 tasks: check if current node is closer than previously found NN, traverse the branch on the same side of the split with respect to the target point, and check if we can prune the other branch (or traverse it as well).

#4 We compute the distance between current node's point and `target`.

#5 If that distance is less than current NN's distance, we have to update the values stored for the NN and its distance.

#6/#7 Check if the target point is on the left branch of the split. If it is, the left branch is the closest to the target point, otherwise it is the furthest.

#8 We certainly need to traverse the closest branch in search of the nearest neighbor. It is important to do so first and update the mementos for NN's distance, to improve pruning.

#9 Using one of the helper functions defined in listing 9.2, we compute the distance between the split line passing through current node and the target point. If this distance is closer than the distance to current nearest neighbor, then the furthest branch might contain points closer than current nearest neighbor (see figure 9.21).